



OFFICIAL PWA & MOBILE ARCHITECTURE MANUAL

The Complete Progressive Web App, Native Engine & Settings Guide

Comprehensive documentation of cassette.fm — a high-fidelity, privacy-preserving music streaming ecosystem. Covering full PWA architecture, YouTube Music InnerTube gateway, lossy & lossless Opus/AAC bitrates, local-first offline caching, two-way Supabase Google Cloud synchronization, and an exhaustive breakdown of every option, subpage, and diagnostic toggle.

• PWA Ready (Service Worker v2)

• Web Audio API & MediaSession

• YouTube Music InnerTube Gateway

• Supabase PostgreSQL & RLS

• React 18 / Vite 8 + Expo Mobile

• Lossless 256kbps Opus (Itag 251)

• Synced LRC Karaoke Engine

• Pure Pitch-Black OLED UI

VERSION

v2.5.0 Production

TARGET PLATFORMS

PWA (Web) & iOS / Android

GENERATED DATE

October 2026

Section 1: System Overview & Design Philosophy	Page 2
Section 2: PWA Architecture, Service Workers & Offline Caching	Page 3
Section 3: Audio Streaming Pipeline & Codec Bitrate Parity	Page 4
Section 4: Core User Views & Interface Walkthrough	Page 5
Section 5: Master Settings Manual — General, Audio & Playback	Page 6
Section 6: Master Settings Manual — Interface, Content & Importers	Page 7
Section 7: Master Settings Manual — Storage, Lyrics, Diagnostics & Hotkeys	Page 8
Section 8: Cloud Database Schema & Two-Way Sync Engine	Page 9
Section 9: Cross-Platform Parity: Web PWA vs Native Mobile App	Page 10
Section 10: Technical Appendix & API Reference Specifications	Page 11

System Overview & Design Philosophy

cassette.fm was created to solve the fundamental compromises of modern commercial streaming services: intrusive subscription paywalls, invasive user telemetry, algorithmically manipulated user feeds, and clunky user interfaces burdened by non-musical fluff.

Pure Pitch-Black Aesthetic (#000000)

Unlike "dark gray" apps that suffer from washouts, cassette.fm forces pure #000000 across canvas, modals, and nav bars. On modern OLED screens, pixels shut off completely, delivering infinite contrast ratios and unmatched battery efficiency.

Color Exclusivity Rule

The interface obeys a strict rule: high-contrast monochrome UI elements (#ffffff and zinc), allowing album artwork to be the single source of vibrant color. An ambient color extraction engine samples the artwork to softly tint docks and backlights.

Custom Retro-Modern Typography

The brand utilizes **Shrikhand**, an expressive 1970s retro display font, paired with clean, geometric **Inter** across song titles, navigation labels, and settings modals, eliminating generic browser fallback styling.

Zero-Compromise Audio Quality

Streams are sourced directly from YouTube Music's high-bitrate Opus (itag 251 at 160–256 kbps, transparent to 320kbps MP3) and AAC pipelines, bypassing compressed 30-second previews.

• CORE PRODUCT GUARANTEES

100% ad-free listening · Zero analytics or user tracking · Local-first offline playback via IndexedDB & Service Worker · Bidirectional Google & Supabase cloud synchronization · Full keyboard navigation & accessibility.

02 PWA Architecture & Offline Foundation

cassette.fm is engineered as a fully compliant, installable **Progressive Web App (PWA)** providing parity with native desktop and mobile applications on macOS, Windows, Linux, iOS, and Android.

PWA Manifest Specification (public/manifest.json)

JSON-LD Compliant

PROPERTY	VALUE	PURPOSE & BEHAVIOR
<code>display</code>	<code>standalone</code>	Hides browser URL bars, tabs, and chrome; opens as an independent native window.
<code>theme_color</code>	<code>#000000</code>	Tints OS system status bars and native title bars to pitch-black.
<code>background_color</code>	<code>#000000</code>	Prevents startup white-flashes while DOM and assets mount.
<code>icons</code>	<code>192x192 & 512x512</code>	High-DPI icons with <code>"purpose": "any maskable"</code> for adaptive Android/iOS home screens.

Service Worker & Multi-Tier Caching Pipeline

The custom Service Worker implements intelligent caching strategies tailored to audio streaming, static application bundles, and dynamic YouTube Music APIs:

1. Cache-First (App Shell)

Applied to HTML, CSS bundles, JS chunks, and Google Font files (Inter, Shrikhand). Serves cached files immediately; updates in background via Stale-While-Revalidate.

2. Network-First (API)

Applied to `/api/search` and `/api/home/feed`. Always attempts live fetch for freshest charts; falls back to cached responses when offline.

3. Persistent Media Store

Downloaded tracks and album art are stored in IndexedDB chunks, allowing full uninterrupted playback without active Wi-Fi or cellular connections.

Web Audio API & Lockscreen Integration

MediaSession API Integration

Wires full playback control to native lockscreens, Bluetooth headphones, smartwatch remotes, and car dashboards. Updates title, artist, album, high-resolution artwork (512×512), and playback position state:

```
navigator.mediaSession.metadata = new
MediaMetadata({
  title: track.title, artist: track.artist,
  artwork: [{ src: track.cover, sizes: '512x512'
}]
});
```

Web Audio Graph & Equalizer

An internal `AudioContext` chains `MediaElementAudioSourceNode` through a `GainNode` (for -14 LUFS loudness normalization and crossfading) into a 128-bin `AnalyserNode` powering the real-time animated equalizer.

03

Audio Streaming Pipeline & Codec Fidelity

Audio delivery is handled through a dual-redundant proxy and resolver engine designed for instantaneous playback start times, seamless seeking, and zero buffering:

YouTube Music Codec & Itag Resolution Matrix					Audiophile Parity
QUALITY PRESET	ITAG	AUDIO CODEC	NOMINAL BITRATE	SAMPLE RATE	CONTAINER
Max (Hi-Fi)	251	Opus (VBR)	160–256 kbps (transparent to 320k MP3)	48.0 kHz	audio/webm; codecs="opus"
High Quality	140	AAC (mp4a.40.2)	128–160 kbps (Clean Studio AAC)	44.1 kHz	audio/mp4
Medium (Data Saver)	249 / 250	Opus (VBR)	50–70 kbps (Ultra-compressed)	48.0 kHz	audio/webm; codecs="opus"

HTTP Range Streaming (RFC 7233)

The streaming server gateway (`/api/stream/:id`) handles partial content streaming via standard HTTP range headers:

- Scrubbing Efficiency:** When users seek forward on the progress bar, the browser requests `Range: bytes=1048576-` , streaming exclusively the remaining bytes without redownloading the track.
- Zero-CORS Embedded Failover:** If direct CDN streams face token expiration or geo-blocks, a headless YouTube audio runtime seamlessly takes over on the client thread, guaranteeing uninterrupted playback.

Dynamic Recommendation & Queue Engine

Watch Next Algorithmic Queue

When any individual song is selected, the server queries `/api/next/:id` , extracting up to 50 contextually related tracks from YouTube Music's recommendation graph. This guarantees automatic, personalized radio without user curation.

Infinite Auto-Advance & Prefetch

When the user reaches the final 2 songs in an active playlist or queue, background workers automatically fetch another recommendation batch using the final song's seed, delivering an endless radio stream.

04 Core Views & Interface Walkthrough

The cassette.fm interface is architected around 5 core view spaces, unified by persistent transport controls and silky-smooth transitions:

1. Dynamic Branded Header

Features the iconic **cassette.fm** brand logo in retro Shrikhand font. Attached is the **Live Status Dot**:

- Emerald: Audio streaming active
- Amber: Cloud synchronization in progress
- Zinc: System idle & ready

On the right are three quick-action icons: **Search Trigger**, **User Avatar / Auth Switcher** (shows Google profile initials when connected), and **Settings Gear**.

2. Home Feed & Horizontal Shelves

Pure real-time feed fetched from `/api/home/feed` organized into horizontal carousel shelves:

- **Filter Chips:** `ALL`, `Quick Picks`, `Mixes`, and `Albums` for instant shelf isolation.
- **Quick Picks:** 16 top chart hits arranged in a 4-row horizontal matrix with thumbnail play and favorite triggers.
- **Daily Mixes:** Algorithmic mood and artist radio mixes with rich artwork.
- **Trending Albums:** Full official releases with dedicated album detail view.
- **Listen Again:** Instant jump-back shelf populated from local playback history.

3. Explore & Search Engine

Debounce real-time search with instant autocomplete suggestions from `/api/search/suggestions`. Implements a **punctuation-normalized search ranking algorithm** so exact track matches (e.g. Steve Lacy's "oh yeah?") always rank above obscure albums. Features multi-category filtering across Songs, Albums, Artists, and Playlists.

4. Library & Custom Playlists

Houses Liked Songs, custom user-created playlists with dynamic 4-tile artwork collages, YouTube Music synced collections, and offline download managers.

5. Floating Mini-Player & Full Sheet

Mounted above tabs with realtime progress line, track metadata, and bouncy spring controls. Tapping opens the Full Sheet with dual Cover Art / Synchronized LRC Lyrics panels.

Master Settings Manual — Account, Audio & Playback

cassette.fm features an exhaustive 9-subpage configuration studio accessible from both the web PWA and mobile applications:

Subpage 1: General & Account Settings

Cloud Auth

1-Click Google OAuth Sign-In

Google OAuth

Authenticates securely via Supabase Google provider; loads profile avatar, name, and email.

Email & Password Auth

Email / Password

Complete support for custom email registrations, sign-in, and password reset flows.

JSON Data Backup & Restore

Export / Import

Export your entire library (liked tracks, playlists, history) to an offline JSON file; import on any device.

Custom Supabase Instance URL

URL & Anon Key

Advanced override allowing power users to hook cassette.fm into self-hosted Supabase backends.

Subpage 2: Audio Engine & Playback Quality

Stream Fidelity

Streaming Audio Quality Preset

Max Hi-Fi

Toggle between Max (256k Opus / Itag 251), High (160k AAC / Itag 140), and Medium (64k Opus / Data Saver).

Audio Volume Normalization

Enabled (Toggle)

Applies ReplayGain loudness leveling (-14 LUFS) to ensure songs play at consistent volume without jarring peaks.

Crossfade Transition Duration

Slider (0-12s)

Smoothly crossfades between consecutive tracks from 0 seconds (off) up to 12 seconds.

Gapless Playback

Enabled (Toggle)

Preloads upcoming track buffers to eliminate dead air between tracks on live and concept albums.

Dynamic Radio Autoplay

Enabled (Toggle)

Automatically queries recommendation API when queue reaches its end to maintain infinite stream.

Remember Last Played Song

Enabled (Toggle)

Restores the last active track and playback position automatically on fresh app launch.

Subpage 3: Appearance, Themes & Interface

Visual Polish

Monochrome B&W Mode

Toggle (Default: Off)

Renders all album art across search, home, and now-playing in high-contrast grayscale.

Dynamic Color Extraction Glow

Dynamic (Toggle)

Samples dominant color palette from album artwork to tint sliders, docks, and glowing borders.

Ambient Background Glow

Toggle (Default: On)

Renders soft, radial ambient backlight behind album cover in the now-playing sheet.

Custom Accent Color Presets

7 Presets

Select fixed UI accent when dynamic glow is off: Pure White, Emerald (#10b981), Amber (#f59e0b), Violet (#8b5cf6), Cyan (#06b6d4), Rose (#f43f5e).

Bottom Tab Visibility (Mobile)

4 Switches

Custom switches to hide or display specific tabs in bottom navigation: Home, Radio, Explore, Library.

Subpage 4: Content & Regional Preferences

Feed Filtering

Content Region / Country Code

Dropdown (ISO 3166)

Sets geographic territory for home feed charts: US, GB, IN, JP, DE, FR, BR, CA, AU, etc.

Explicit Content Filter

Toggle (Default: Off)

SafeMode switch to filter out tracks with explicit language badges from search and recommendations.

Music Language Preferences

Multi-select Chips

Prioritizes regional language charts (English, Hindi, Punjabi, Tamil, Telugu, Spanish, etc.).

Subpage 5: External Playlist Importers

Data Portability

YouTube Public Playlist Importer

URL Input + Fetch

Paste any public or unlisted YouTube Music playlist URL; imports all tracks into a native playlist instantly.

Spotify Playlist Importer

Spotify Bridge

Resolves Spotify tracklists via Spotify Web API credits endpoint and matches each track to high-bitrate YouTube streams.

07

Master Settings Manual — Storage, Lyrics, Diagnostics & Hotkeys

Subpage 6: Storage, Downloads & Offline Mode

Cache Manager

Storage Footprint Inspector

Live MB Counter

Displays live breakdown of IndexedDB cached audio tracks, artwork image blobs, and offline state.

Clear Cache & Re-index

Purge Cache

Purges temporary audio buffers and stale search cache while strictly safeguarding your liked songs & playlists.

Subpage 7: Synchronized Lyrics Engine

LRC Karaoke

Synchronized Karaoke Lyrics

Enabled

Parses timestamped LRC files; highlights the currently sung lyric line with auto-scrolling and tap-to-seek.

Lyrics Typography Size

4 Size Tiers

Select text scaling for lyric display: Small (16px), Normal (20px), Large (24px), Extra Large (28px).

Romanized Transliteration

Toggle (Romanized)

Generates phonetic Latin script transliteration for songs with Japanese Kanji/Kana, Korean Hangul, or Cyrillic.

Subpage 8: Diagnostics & Developer Tools

Telemetry & Health

Supabase Database Ping

Ping Cloud

Tests cloud latency (ms) and validates PostgreSQL Row Level Security credentials.

Audio Stream Resolver Health

Itag Inspector

Inspects active audio stream itag (e.g. 251 Opus), container type, sample rate, and buffering latency.

Live Diagnostics Event Log

Copy Logs

Streams real-time internal errors, network status codes, and cache events with one-click copy to clipboard.

Subpage 9: Keyboard Shortcuts Reference Matrix (PWA)

Accessibility

SHORTCUT KEY	ACTION EXECUTED	SHORTCUT KEY	ACTION EXECUTED
Space	Play / Pause Toggle	Shift + →	Skip to Next Track
Shift + ←	Skip to Previous Track	L	Toggle Favorite / Liked Song
M	Mute / Unmute Audio	/ or F	Focus Real-Time Search Bar
Esc	Close Modal / Back to Home	Shift + S	Toggle Shuffle Mode

08 Cloud Database Schema & Two-Way Sync Engine

Multi-device synchronization is powered by **Supabase PostgreSQL** with strict **Row Level Security (RLS)**, ensuring users can only read and write their own data:

PostgreSQL Tables Structure (supabase_schema.sql)

Postgres 15

TABLE NAME	COLUMNS & DATA TYPES	SECURITY & PURPOSE
profiles	id (uuid PK), full_name (text), email (text), avatar_url (text), updated_at	Stores user metadata; auto-populated upon Google OAuth login via PostgreSQL trigger.
liked_songs	id (uuid PK), user_id (uuid FK), song_id (text), song_data (jsonb), created_at	Unique index on (user_id, song_id). Stores full track metadata payload for instant offline hydration.
user_playlists	id (text PK), user_id (uuid FK), name (text), songs (jsonb[]), created_at, updated_at	Stores custom playlists created on Web or Mobile. Allows collaborative sharing when marked public.
playback_history	id (uuid PK), user_id (uuid FK), song_data (jsonb), played_at (timestampz)	Chronological listening stream powering the "Listen Again" home shelf across all logged-in devices.

Two-Way Union Merge Algorithm

When users sync between devices or link their Google accounts, cassette.fm executes a conflict-free 4-phase synchronization algorithm:

Phase 1: Local Push

Extracts local `likedSongs` and `pulse_playlists` from storage; pushes newly created local items up to Supabase Cloud using batch `upsert`.

Phase 2: Cloud Pull

Retrieves cloud-stored liked songs, custom playlists, and playback history records for the authenticated `user_id`.

Phase 3: Union Reconciliation

Merges local and cloud collections by unique `id / videoId`. If playlists conflict, the version with the greatest track count or newest timestamp wins.

Phase 4: Persistence

Writes merged union back to local storage and pushes merged deltas to the cloud database, achieving 100% parity across devices.

09 Cross-Platform Parity: Web PWA vs Native Mobile App

cassette.fm shares a unified architectural core (`@cassette/core`) between the Web PWA client and the native iOS/Android Expo client:

Feature & Implementation Parity Matrix

Full Ecosystem Parity

CAPABILITY / FEATURE	WEB PWA IMPLEMENTATION	NATIVE MOBILE IMPLEMENTATION
Audio Playback Engine	Web Audio API + HTML5 Audio + AnalyserNode	<code>react-native-track-player</code> Native Foreground Service
Background Playback	MediaSession API + Service Worker worker thread	Android MediaStyle Notification + iOS Audio Background Mode
Startup Splash Experience	CSS pure black canvas (#000000)	Expo Splash Plugin + <code>preventAutoHideAsync()</code>
UI Gestures & Animations	CSS transitions + Tailwind utility classes	<code>react-native-reanimated</code> Spring & Timing physics
State Management	React Context + Zustand stores	Zustand stores (shared isomorphic <code>@cassette/core</code>)
Offline Data Storage	IndexedDB + LocalStorage	<code>AsyncStorage</code> + SQLite offline cache
Navigation Layout	Desktop Sidebar / Top Bar	4-Tab Bottom Nav (Home, Radio, Explore, Library)
Settings Access	Header gear icon & modal studio	Top-right header gear icon strictly

Monorepo Code Organization

```
cassette.fm/ monorepo
├─ src/                # Web Progressive Web App (React 18 / Vite 8 / Tailwind)
├─ apps/mobile/        # Native Mobile Client (Expo SDK 52 / React Native / Reanimated)
├─ packages/core/      # Shared Isomorphic Logic (Zustand stores, Supabase, InnerTube)
├─ api/                # Serverless API Endpoints (Stream proxy, search, feeds)
└─ supabase_schema.sql # PostgreSQL schema & Row Level Security definitions
```

10 Technical Appendix & API Reference Specifications

cassette.fm communicates through high-speed serverless endpoints connecting client apps to YouTube Music and Supabase:

Serverless API Endpoints Specification

RESTful JSON Gateway

ENDPOINT ROUTE	HTTP METHOD	REQUEST QUERY / PAYLOAD	RESPONSE DESCRIPTION
<code>/api/home/feed</code>	GET	None	Returns live Quick Picks, Daily Mixes, and Trending Albums shelves.
<code>/api/search</code>	GET	<code>?q={query}&type={songs albums ...}</code>	Performs punctuation-normalized search across YouTube Music catalog.
<code>/api/search/suggestions</code>	GET	<code>?q={query}</code>	Live autocomplete suggestions as the user types in the search bar.
<code>/api/stream/:id</code>	GET	<code>Range: bytes=...</code>	Streams high-bitrate Opus (Itag 251) or AAC with 206 Partial Content.
<code>/api/next/:id</code>	GET	None	Extracts 50 Watch Next algorithmic recommendations from seed track.
<code>/api/lyrics</code>	GET	<code>?id={videoId}&title={t}&artist={a}</code>	Retrieves synchronized LRC lyrics with millisecond timestamp markers.
<code>/api/spotify/credits</code>	GET	<code>?q={songTitle}+{artist}</code>	Enriches track with Spotify songwriter, producer, and performer credits.
<code>/api/ytmusic/library</code>	POST	<code>{ cookie?: string }</code>	Syncs user's remote YouTube Music playlists and liked tracks.
<code>/api/ytmusic/like</code>	POST	<code>{ videoId, liked: boolean }</code>	Dispatches like / unlike events back to YouTube Music cloud account.

Summary & Conclusion

cassette.fm represents a gold standard in modern Progressive Web App engineering: fusing uncompromised audiophile playback (256kbps Opus), tactile 1970s cassette aesthetics with pitch-black OLED styling, instant offline responsiveness, and full cloud data sovereignty.

cassette.fm · Engineered with React, Vite, InnerTube, Supabase & Web Audio API. Zero Ads · Zero Telemetry.